

Author

Priya Dharshini V

24f2003684

24f2003684@ds.study.iitm.ac.in

I am a dual degree student pursuing B.Tech in CSE (AI & ML) at VIT Chennai and the BS in Data Science from IIT Madras. I'm interested in problem solving and full stack development.

Description

The Vehicle Parking app SafePark is a multi-user app which allows two kinds of users admin and user to do their respective roles:

Admin: To view, add and delete lots and spots and manage users and reservations.

User: To book and release spots, to view their history and summary.

Approximately 5% of AI assistance was used in this project, mainly for error detection, improving code readability, and generating basic documentation structure.

Technologies used

Frontend: HTML, CSS, Bootstrap, ChartJS

- HTML was used to create forms for login, registration, lot creation, and reservation input.
- CSS and Bootstrap styled the pages and made them responsive, ensuring users could navigate easily on any device.
- Bootstrap components like navbars, modals, buttons, and cards were used to organize and display information clearly.
- ChartJS visualized data like the number of reservations and revenue through bar and doughnut charts in the summary dashboard.

Backend: Flask, Flask-Login, Flask-SQLAlchemy

- Flask handled routing between pages (e.g., /login, /reserve, /admin/summary) and managed HTTP requests.
- Flask-Login was used to implement secure login and logout, track logged-in users, and restrict access to admin/user pages.
- Flask-SQLAlchemy defined models for users, parking lots, spots, and reservations, allowing us to perform operations like querying reservations, updating statuses, and calculating earnings with Python code.

DB Schema Design

The application uses a relational database with four core tables: User, Parking_lot, Parking_spot, and Reservation. The schema is designed using SQLAlchemy in Flask. Below is the breakdown:

- User
 - id (PK, Auto-Increment) – Unique user ID
 - name, email (unique, not null), address, username (unique, not null)
 - password (hashed, not null), role (default: user)
 - pin_code (not null)
 - Linked to reservations (One-to-Many)
- Parking_lot
 - id (PK), lot_location, address, pin_code, price, max_spots (all not null)
 - total_price (default: 0) – Tracks total earnings
 - Linked to parking spots and reservations
- Parking_spot
 - id (PK) – Custom ID format
 - lot_id (FK to Parking_lot)
 - status (default: active)
 - Linked to one lot and many reservations
- Reservation
 - id (PK, auto), spot_id, lot_id, customer_id (FKs)
 - vehicle_no (not null)
 - start_time (default: current time), end_time
 - price_earned, status (default: occupied)

API Design

This project primarily uses Flask routes to handle user registration, login, parking lot management, and spot reservations. While the app is designed for server-side rendering, it follows REST-like principles with multiple **GET** and **POST** endpoints. Functionalities include user authentication, booking and releasing parking spots, viewing history, and admin-level lot/spot management. Form data is handled using **request.form**, and route-based access control is enforced with **Flask-Login**.

Architecture and Features

The project follows a modular structure for clarity and maintainability:

Project Structure:

- controllers/ contains route handlers, including login_controller.py for authentication and role-based login logic.
- models/ holds the SQLAlchemy database models in model.py, defining all tables and their relationships.

- static/ stores static files like CSS (dashboard.css, home.css) and images (icon.png).
- templates/ has subfolders (admin, user, home, base) with HTML pages rendered for different roles and functionalities using Jinja templating.
- app.py serves as the main Flask entry point, initializing routes, extensions, and registering Blueprints.
- seed_data.py is used to populate the database with initial test data.

Features Implemented:

- User Authentication: Login and logout handled through Flask-Login. Passwords are hashed using werkzeug.security.
- Role-Based Access: Admins and users see different dashboards and functionalities.
- Parking Lot and Spot Management: Admins can add, view, and manage parking lots and their spots.
- Reservation System: Users can book and release parking spots with status tracking (occupied, released).
- Dashboard & Summary Views: Visual representations (e.g., Chart.js) of reservations, along with earnings.
- Responsive UI: Templates are structured using reusable components and styled with custom CSS.

Video

<https://drive.google.com/file/d/1YsJsWpvMvXp2S7FDe5jKb21bJpcDYyIrl/view?usp=sharing>